

Python Objects

Object Oriented

- A program is made up of many cooperating components
- We can organize these components into different “objects” that each do some work

Object

- An Object is a bit of self-contained Code and Data
- A key aspect of the Object approach is to break the problem into smaller understandable parts (divide and conquer)
- We have been using objects all along: String Objects, Integer Objects, Dictionary Objects, List Objects...

Definitions

- **Class** - a template
- **Method or Message** - A defined capability of a class
- **Field or attribute** - A bit of data in a class
- **Object or Instance** - A particular instance of a class

A Sample Class



class is a reserved word that defines a template for making objects

Each PartyAnimal object has a bit of code

Tell the `an` object to run the `party()` code within it

```
class PartyAnimal:

    def __init__(self):
        self.x = 0

    def party(self) :
        self.x = self.x + 1
        print("So far",self.x)

an = PartyAnimal()

an.party()
an.party()
an.party()
```

In `an = PartyAnimal()`, the special `__init__` method is called to create the object.

Each `an.party()` will call `PartyAnimal's party()` method, and increment the class variable `x`

Running the code:

```
class PartyAnimal:

    def __init__(self):
        self.x = 0

    def party(self) :
        self.x = self.x + 1
        print("So far",self.x)

an = PartyAnimal()

an.party()
an.party()
an.party()
```

```
class PartyAnimal:

    def __init__(self):
        self.x = 0

    def party(self) :
        self.x = self.x + 1
        print("So far",self.x)

an = PartyAnimal()

an.party()
an.party()
an.party()
```

Running the code:

So far 1

So far 2

So far 3

Object Lifecycle

- Objects are created, used, and discarded
- We have special blocks of code (methods) that get called
 - At the moment of creation (constructor)
 - At the moment of destruction (destructor)
- Constructors (the code in the `__init__`) are used a lot
- Destructors are not used often

Constructor

The primary purpose of the constructor is to set up some instance variables to have the proper initial values when the object is created

```
class PartyAnimal:

    def __init__(self):
        self.x = 0
        print('I am constructed')

    def party(self) :
        self.x = self.x + 1
        print('So far',self.x)

    def __del__(self):
        print('I am destructed', self.x)

an = PartyAnimal()
an.party()
an.party()
an = 42
print('an contains',an)
```

Running the code:

```
I am constructed
So far 1
So far 2
I am destructed 2
an contains 42
```

The constructor and destructor are optional. The constructor is typically used to set up variables. The destructor is seldom used.

```
class PartyAnimal:

    def __init__(self):
        self.x = 0
        print('I am constructed')

    def party(self) :
        self.x = self.x + 1
        print('So far',self.x)

    def __del__(self):
        print('I am destructed', self.x)

an = PartyAnimal()
an.party()
an.party()
an = 42
print('an contains',an)
```

Running the code:

```
I am constructed
So far 1
So far 2
I am destructed 2
an contains 42
```

The constructor and destructor are optional. The constructor is typically used to set up variables. The destructor is seldom used.

```
class PartyAnimal:

    def __init__(self):
        self.x = 0
        print('I am constructed')

    def party(self) :
        self.x = self.x + 1
        print('So far',self.x)

    def __del__(self):
        print('I am destructed', self.x)

an = PartyAnimal()
an.party()
an.party()
an = 42
print('an contains',an)
```

Running the code:

I am constructed
So far 1
So far 2
I am destructed 2
an contains 42

The constructor and destructor are optional. The constructor is typically used to set up variables. The destructor is seldom used.

```
class PartyAnimal:

    def __init__(self):
        self.x = 0
        print('I am constructed')

    def party(self) :
        self.x = self.x + 1
        print('So far',self.x)

    def __del__(self):
        print('I am destructed', self.x)

an = PartyAnimal()
an.party()
an.party()
an = 42
print('an contains',an)
```

Running the code:

I am constructed
So far 1
So far 2
I am destructed 2
an contains 42

The constructor and destructor are optional. The constructor is typically used to set up variables. The destructor is seldom used.

```
class PartyAnimal:

    def __init__(self):
        self.x = 0
        print('I am constructed')

    def party(self) :
        self.x = self.x + 1
        print('So far',self.x)

    def __del__(self):
        print('I am destructed', self.x)

an = PartyAnimal()
an.party()
an.party()
an = 42
print('an contains',an)
```

Running the code:

I am constructed
So far 1
So far 2
I am destructed 2
an contains 42

The constructor and destructor are optional. The constructor is typically used to set up variables. The destructor is seldom used.

```
class PartyAnimal:

    def __init__(self):
        self.x = 0
        print('I am constructed')

    def party(self) :
        self.x = self.x + 1
        print('So far',self.x)

    def __del__(self):
        print('I am destructed', self.x)

an = PartyAnimal()
an.party()
an.party()
an = 42
print('an contains',an)
```

Running the code:

I am constructed
So far 1
So far 2
I am destructed 2
an contains 42

The constructor and destructor are optional. The constructor is typically used to set up variables. The destructor is seldom used.

Many Instances

- We can create **lots of objects** - the class is the template for the object
- We can store each **distinct object** in its own variable
- We call this having multiple **instances** of the same class
- Each **instance** has its own copy of the **instance variables**

```
class PartyAnimal:

    def __init__(self, z):
        self.x = 0
        self.name = z
        print(self.name, "constructed")

    def party(self) :
        self.x = self.x + 1
        print(self.name, "party count", self.x)
```

```
s = PartyAnimal("Sally")
```

Instance 1

```
s.party()
```

```
j = PartyAnimal("Jim")
```

Instance 2

```
j.party()
```

```
s.party()
```

Constructors can have additional parameters.

These can be used to set up instance variables for the particular instance of the class (i.e., for the particular object).

```
class PartyAnimal:
```

```
    def __init__(self, z):  
        self.x = 0  
        self.name = z  
        print(self.name, "constructed")
```

```
    def party(self) :  
        self.x = self.x + 1  
        print(self.name, "party count", self.x)
```

```
s = PartyAnimal("Sally")
```

```
s.party()
```

```
j = PartyAnimal("Jim")
```

```
j.party()
```

```
s.party()
```

```
Sally constructed  
Sally party count 1  
Jim constructed  
Jim party count 1  
Sally party count 2
```

Inheritance

Inheritance

- When we make a new class - we can reuse an existing class and **inherit** all the capabilities of an existing class

```
class PartyAnimal:
```

```
    def __init__(self, nam):  
        self.x = 0  
        self.name = nam  
        print(self.name, "constructed")
```

```
    def party(self) :  
        self.x = self.x + 1  
        print(self.name, "party count", self.x)
```

```
class FootballFan(PartyAnimal):
```

```
    def __init__(self, nam) :  
        super().__init__(nam)  
        self.points = 0
```

```
    def touchdown(self):  
        self.points = self.points + 7  
        self.party()  
        print(self.name, "points", self.points)
```

FootballFan *inherits* from **PartyAnimal**.

It has:

1. All the capabilities of **PartyAnimal**
2. Some code just for itself (the method touchdown)

```
class PartyAnimal:
```

```
    def __init__(self, nam):  
        self.x = 0  
        self.name = nam  
        print(self.name,"constructed")  
  
    def party(self) :  
        self.x = self.x + 1  
        print(self.name,"party count",self.x)
```

```
class FootballFan(PartyAnimal):
```

```
    def __init__(self, nam) :  
        super().__init__(nam)  
        self.points = 0  
  
    def touchdown(self):  
        self.points = self.points + 7  
        self.party()  
        print(self.name,"points",self.points)
```

```
s = PartyAnimal("Sally")  
s.party()
```

```
j = FootballFan("Jim")  
j.party()  
j.touchdown()
```

Running the code above:

```
Sally constructed  
Sally party count 1  
Jim constructed  
Jim party count 1  
Jim points 7
```

Definitions

- **Class** - a template
- **Attribute** – A variable within a class
- **Method** - A function within a class
- **Object** - A particular instance of a class
- **Constructor** – Code that runs when an object is created
- **Inheritance** - The ability to extend a class to make a new class.

Summary

- Object Oriented programming is a very structured approach to code reuse
- We can group data and functionality together and create many independent instances of a class



Acknowledgements / Contributions



These slides are Copyright 2010- Charles R. Severance (www.dr-chuck.com) of the University of Michigan School of Information and made available under a Creative Commons Attribution 4.0 License. Please maintain this last slide in all copies of the document to comply with the attribution requirements of the license. If you make a change, feel free to add your name and organization to the list of contributors on this page as you republish the materials.

Initial Development: Charles Severance, University of Michigan School of Information

Adapted by Minh Tran, University of Chicago, Department of Computer Science for use in the pre-college course Creating Computer Games for Learning (Summer 2026).